

PostgreSQL

So your Postgres server has run out of disk space

- Don't panic – Postgres should handle the situation cleanly
- Check `tail /var/log/gitlab/postgresql/current`
- Check `select * from pg_replication_slots`
- Check `du -sh /var/opt/gitlab/postgresql/data/pg_xlog`

In fact often Postgres will just generate regular transaction ERRORS and continue without crashing at all. This is especially true if it's mainly running read-only queries. When there is disk space available again it will continue as if nothing happened. So if the server is still up focus on adding space or finding the log files or other files consuming space.

If it cannot write XLOG then it will crash. The log files will show a PANIC log message. But when the disk space is free again it should start up and do regular crash recovery so still focus on adding space or identifying the top consumer of space and cleaning it up.

- If the `pg_xlog` (or `pg_wal` in PG 11+) directory is the problem:

If there's a replication slot that has `active` false and has a markedly old `restart_lsn` then it may be forcing Postgres to keep lots of old xlog around.

If the replica cannot be resurrected (or if there are other healthy replicas and you would rather get the primary back sooner and lose this one) then run `select pg_drop_replication_slot('slot_name');`

If the server has already crashed then the best strategy is to find a few gigabytes to remove, start it up and then run `pg_drop_replication_slot` as above. On our servers there is a file `DO_NOT_MOVE_THIS_FILE` which you can remove to create space to do this.

Failing that you may have to remove old xlog files manually. This is extremely hazardous as if you remove a needed file the server will be lost and unable to recover.

You can check the log files for the last few checkpoint logs or you can run `pg_controldata`:

```
$ /opt/gitlab/embedded/postgresql/bin/pg_controldata /var/opt/gitlab/postgresql/data
```

Latest checkpoint location:	2B78/E30A3218
Prior checkpoint location:	2B78/DD37A880
Latest checkpoint's REDO location:	2B78/DF728AB8
Latest checkpoint's REDO WAL file:	0000000400002B78000000DF

WARNING: Do not under any circumstances remove any xlog files newer than these xlog locations. The “Latest checkpoint’s REDO WAL file” is the simplest one to follow as it’s the actual file name. However ideally you should retain more than that. Just remove a minimum number of the oldest files.

WARNING: Do not remove any files that end in `.history`. These files are needed for standbys to follow the wal through timeline forks. A new standby will not start up if this file is not present.

Reconstruct a standby after the xlog has been pruned on the primary

If you see log messages like:

```
2018-01-29_10:13:01.21015 postgres02 postgresql: 2018-01-29 10:13:01 GMT [91066]: [1-1] LOG:
2018-01-29_10:13:01.21047 postgres02 postgresql: 2018-01-29 10:13:01 GMT [91066]: [2-1] FATAL: 
```

Then the standby is lost due to the xlog on the primary being pruned.

If the standby is properly configured it should be able to retrieve the archived logs from blob storage. Currently this requires installing the WAL-E PGP private key in the agent in order for that to work.

- Start a tmux session – the standby clone will take several hours
- Ensure you’re on the correct replica host!
- Ensure the database (patroni service) is shut down on the replica
- `watch du -sh /var/opt/gitlab/postgresql/data`
- `tail -f /var/log/gitlab/postgresql/current`
- `rm -r /var/opt/gitlab/postgresql/data/*`
- Follow `patroni-management.md` to bootstrap the replica

While the server is starting up – especially for the first time – it is normal to see many log entries like this which simply indicate that a client has tried to connect before the server is finished starting up:

```
2018-01-30_01:00:09.80433 postgres02 postgresql: 2018-01-30 01:00:09 GMT [110492]: [1-1] FATAL: 
```

When it’s ready you should see log entries like this when the database starts up:

```
2018-01-30_01:00:11.58219 postgres02 postgresql: 2018-01-30 01:00:11 GMT [110184]: [4-1] LOG:
2018-01-30_01:00:11.58246 postgres02 postgresql: 2018-01-30 01:00:11 GMT [110182]: [1-1] LOG:
2018-01-30_01:00:11.65525 postgres02 postgresql: 2018-01-30 01:00:11 GMT [110525]: [1-1] LOG: 
```